# Trabajo Práctico 1: Metaprogramación

Traits en Ruby

## Objetivos y recomendaciones

El objetivo principal de este TP es el de introducirnos en el mundo de la metaprogramación aplicada a una idea particular. Gran parte de la resolución consiste en lograr construir abstracciones entendiendo que estamos creando una herramienta genérica, que podría utilizarse como parte de modelos de cualquier dominio.

El TP requiere la aplicación de mecanismos específicos del lenguaje asociados a la metaprogramación: introspección e intercesión, pero además de conceptos de diseño orientado a objetos. En otras palabras, no tienen que olvidarse de los lineamientos para un buen diseño que vienen usando hasta ahora durante la carrera (y para otros dominios). Por supuesto, como siempre, la solución deberá estar acompañada de pruebas automatizadas.

## Descripción general del Dominio

Se pide implementar un mecanismo de composición de objetos similar al de *Traits*, con su álgebra incluida, basándose en el artículo *"Traits: A Mechanism for Fine-grained Reuse"*[1], de Ducasse, Nierstrasz, Schärli, Wuyts y Black. Además de leer este documento, también deben leer el paper para entender bien de qué se trata y tener una definición formal de *trait*[2].

A continuación vamos a enumerar algunos requisitos funcionales. Forma parte del TP que consulten sobre todos los aspectos del dominio que no estén explícitos o que queden ambiguos en este documento (como ocurre con cualquier otro dominio, aún cuando este es un dominio computacional).

---

[1] https://web.cecs.pdx.edu/~black/publications/TOPLASTraits.pdf

[2] Lo más importante para lo que pedimos en el TP es la sección 4 del artículo, aunque les recomendamos leerlo completo.

A partir de la sección 4.1 se describe un modelo formal de lo que es un trait y de las operaciones que conforman el álgebra de traits. Esta formalización puede ser muy útil como *especificación*. Sin embargo, durante el TP no vamos a estar trabajando en términos de este formalismo.

Por otra parte, no vamos a seguir *todo* lo que plantea el artículo *al pie de la letra*. Por ejemplo, debido a que las variables de instancia son dinámicas en Ruby, podremos definir traits con estado sin ninguna dificultad (que, además, es una extensión del modelo que se hizo más adelante; para ver esto pueden consultar el artículo *A new modular implementation for Stateful Traits, 2020*).

Cualquier duda que tengan sobre este tipo de cosas, avisen y las charlamos en clase.

Además, dado que estaremos desarrollando una herramienta para programar, deberían estar en condiciones de hacer sugerencias sobre el dominio. En ese sentido, todo lo que se pide en este documento puede cuestionarse y mejorarse, siempre y cuándo den buenas razones para hacerlo *y haya una conversación y un acuerdo con los docentes*. Es decir, pueden hacer cualquier sugerencia para mejorar la herramienta que estamos creando, incluso cuando implique hacer algo distinto a lo que se pide explícitamente. Que tengan un buen criterio para plantear estas mejoras es parte de lo que tomamos en cuenta para la evaluación del TP.

## Definición y aplicación de traits

El primer requisito es poder definir un *trait* e incorporarlo a una clase. Nos gustaría poder definir un trait de la siguiente manera:

```
trait Atacante do
    def ataque
        10
    end
end
```

Y luego poder usarlo de esta forma:

```
class Fantasma
    uses Atacante

    def nombre(sufijo)
        "Casper" + sufijo
    end
end
```

Ahora, si tratamos de usar un objeto de tipo `Fantasma`, debería ocurrir lo siguiente:

```
fantasma = Fantasma.new
fantasma.ataque      # => 10
fantasma.nombre("!") # => "Casper!"
```

## Posibles conflictos

Supongamos que tenemos la siguiente clase, que usa el trait `Atacante` que definimos anteriormente:

```
class Fantasma
    def ataque
        20
    end
```

```
    uses Atacante
end
```

Luego, cuando le enviamos el mensaje `ataque` a una instancia de `Fantasma`, lo que debería ocurrir es lo siguiente:

```
fantasma = Fantasma.new
fantasma.ataque # => 20
```

Es decir, se deberían agregar a la clase los métodos definidos en el trait, pero sin pisar los métodos que ya estén definidos en la clase. Esto se puede ver en el hecho de que, si bien el trait define `ataque`, la clase también, y "gana" la implementación de la clase.

## Métodos requeridos

Un trait puede especificar que requiere algunos métodos adicionales no definidos por él. Por ejemplo:

```
trait Humano do
    requires :nombre, :apellido

    def saludo
        "Hola, soy #{self.nombre} #{self.apellido}"
    end
end
```

Esto significa que esperamos que las clases que usen este trait tengan definidos los métodos `nombre` y `apellido` (ya sea porque los definió la propia clase, o porque los obtuvo a partir de otros traits).

Por ejemplo, la siguiente clase cumple con estos requisitos, ya que define ambos métodos:

```
class Ciudadano
    def nombre
        "Juan"
    end

    uses Humano

    def apellido
        "Pérez"
    end
end

Ciudadano.new.saludo # => "Hola, soy Juan Pérez"
```

En caso de que un método requerido no esté definido, se deberá levantar una excepción al recibir el mensaje correspondiente:

```
class Ciudadano
    uses Humano
end

Ciudadano.new.saludo # ¡Excepción!
```

📝 *Nota*: si un método requerido está definido en la jerarquía de la clase que usa los traits, esto cuenta para satisfacer el requerimiento:

```
class ConNombre
    def nombre
        "Juan"
    end
    def apellido
        "Pérez"
    end
end

class Ciudadano < ConNombre
    uses Humano
end

Ciudadano.new.saludo # => "Hola, soy Juan Pérez"
```

# Operaciones

## Composición de Traits

Se desea ahora agregar la operación de *combinación simétrica* (o suma) de traits. Esta operación debe permitir combinar dos traits; al usar su resultado, se agregan a la clase los métodos de los traits que se combinaron:

```
trait Atacante do
    def ataque
        10
    end
end

trait Defensor do
    def defensa
        20
```

```
    end
end

class Guerrero
    uses Atacante + Defensor
end

guerrero = Guerrero.new
guerrero.ataque     # => 10
guerrero.defensa    # => 20
```

La operación de composición de traits debe ser *conmutativa*, *asociativa* e *idempotente*.

### Posibles conflictos

Un conflicto surge cuando se componen traits que tienen métodos para el mismo mensaje. Por ejemplo:

```
trait Atacante do
    def recuperarse
        "recuperarse como Atacante"
    end
end

trait Defensor do
    def recuperarse
        "recuperarse como Defensor"
    end
end

class Guerrero
    uses Atacante + Defensor # ¡Excepción!
end
```

La excepción a esto es cuando, si bien hay más de un método con el mismo nombre, son *el mismo* método (es decir, vienen del mismo trait). Por ejemplo:

```
trait ConNombre do
    def nombre
        "Atila"
    end
end

class Guerrero
    uses Atacante + Atacante # No debería levantar una excepción, es
```

```
                                # equivalente a usar solo Atacante
end
```

Otro ejemplo:

```
class Guerrero
    uses (Atacante + ConNombre) +
         (ConNombre + Atacante) # No debería levantar una excepción, es
                                # equivalente a usar Atacante+ConNombre
end
```

El comportamiento en estos casos particulares es consecuencia necesaria de las propiedades que dijimos que tenía que tener la suma de traits.

### Interacción con métodos requeridos

Si hay métodos requeridos, éstos pueden satisfacerse al componer el trait que los requiere con otros que definan esos métodos. Por ejemplo:

```
trait Humano do
    requires :nombre, :apellido

    def saludo
        "Hola, soy #{self.nombre} #{self.apellido}"
    end
end

trait ConNombre do
    def nombre
        "Juan"
    end
end

trait ConApellido do
    def apellido
        "Pérez"
    end
end

class Ciudadano
    uses ConNombre + Humano + ConApellido
end

Ciudadano.new.saludo # => "Hola, soy Juan Pérez"
```

## Excluir un método de un trait

La siguiente operación nos permite obtener un nuevo trait que sea igual a otro pero excluyendo la implementación para un selector en particular.
Utilizando esta operación, por ejemplo, se podría resolver un conflicto de la siguiente manera:

```
class Guerrero
    uses Atacante + (Defensor - :recuperarse)
end

guerrero = Guerrero.new
guerrero.recuperarse   # => "recuperarse como Atacante"
```

O, alternativamente, si no quisiéramos ninguna implementación para recuperarse

```
class Guerrero
    uses (Atacante + Defensor) - :recuperarse
end

guerrero = Guerrero.new
guerrero.respond_to?(:recuperarse) # => false
```

### Soporte para excluir múltiples métodos

Queremos también poder excluir más de un método a la vez. Para especificar esto en forma más concisa, tenemos que poder hacer lo siguiente:

```
UnTrait - [:metodo1, :metodo2, :metodo3]
```

Por supuesto, también podría hacerse utilizando múltiples veces la operación de exclusión:

```
UnTrait - :metodo1 - :metodo2 - :metodo3
```

## Definición de alias de métodos

Por último, se pide tener la operación de “alias” para poder ponerle un nombre alternativo adicional a un método. Es decir, que se incluya otro mensaje para el mismo método, con el nombre que nosotros especificamos. Esta operación nos permite, por ejemplo, poder resolver los conflictos sin perder acceso a los métodos conflictivos.

Veamos cómo resolver el caso anterior, en el cual queremos implementar `recuperarse` invocando las dos implementaciones de los traits `Atacante` y `Defensor`

```
class Guerrero
   uses
     ((Atacante << {recuperarse: :recuperarse_como_atacante}) - :recuperarse)
     +
     ((Defensor << {recuperarse: :recuperarse_como_defensor}) - :recuperarse)
```

```
  def recuperarse
      recuperarse_como_atacante + " y " + recuperarse_como_defensor
  end
end

guerrero = Guerrero.new

guerrero.recuperarse
# => "recuperarse como Atacante y recuperarse como Defensor"
```

## Estrategias de resolución de conflictos

Además de las operaciones existentes, se desea tener distintas estrategias ya programadas para resolver conflictos. Estas estrategias *deben definirse por cada mensaje con métodos conflictivos*.

Las estrategias de resolución a implementar son:

1. Que, en caso de haber conflicto entre dos métodos de traits al combinarse, se tome automáticamente el método definido por uno de los traits.
2. Que ejecute todos los métodos conflictivos.
   Por ejemplo, en el caso de combinar `Atacante` con `Defensor`, querríamos que al enviar el mensaje `recuperarse` a una instancia de la clase que use el trait resultante, se ejecute el método `recuperarse` definido en `Atacante` y el método `recuperarse` definido en `Defensor`.
3. Hacer un *inject/reduce*, para combinar el resultado de haber evaluado los métodos conflictivos. Para esto, tenemos que recibir como argumentos un valor inicial y una función que cumpla el rol de combinar un acumulador con cada uno de los resultados de los métodos.
   Por ejemplo, supongamos que tenemos un Trait`A` que define `m1` y retorna un número, y otro Trait`B` que también define `m1` y retorna otro número, queremos que al combinarlos “se sumen” ambos resultados. Entonces, para combinarlos, pasamos al cero (`0`) como valor inicial, y la función de suma (`{|acc, n| acc + n }`).
4. Que se permita la definición de estrategias adicionales al programador, además de las ya definidas. Forma parte del TP que propongan una interfaz apropiada para poder realizar esto.

## Protocolo de reflexión

Forma parte del TP que piensen en un protocolo de reflexión/introspección adecuado para los traits. Este protocolo, además de poder ser usado por las personas que programan usando traits, puede ser una buena interfaz en base a la cual escribir las pruebas de las

reglas del dominio. Sería deseable, también, considerar las convenciones que usa Ruby en su metamodelo, para que sea un agregado cohesivo.

Dejamos algunas sugerencias aquí:

- podría ser interesante consultar por los métodos que define el trait para las instancias de las clases que usen el trait, por ejemplo:

```
trait Atacante do
    def ataque
        10
    end
end

Atacante.instance_methods # => [:ataque]
Atacante.instance_method(:ataque) #=> #<UnboundMethod: ...#ataque>
```

- o preguntarle a una clase cuáles son los traits que usa:

```
class Guerrero
    uses Atacante
end
Guerrero.traits #=> [Atacante]
```

  y que también funcione, por supuesto, con traits resultantes de operaciones:

```
class Guerrero
    uses Atacante + Defensor
end
Guerrero.traits #=> [Atacante + Defensor]
```

---

# Puntos Bonus

## Uso de traits por otros traits

Los traits pueden usar otros traits. Por ejemplo:

```
trait ConNombre do
    def nombre
        "Atila"
    end
end
```

```
trait Atacante do
    uses ConNombre
    def ataque
        10
    end
end

class Guerrero
    uses Atacante
end

atila = Guerrero.new
atila.ataque # => 10
atila.nombre # => "Atila"
```

Queda a criterio del grupo evaluar todos los casos bordes con respecto a esto, y el planteamiento de un modelo lo más uniforme posible (y con menos sorpresas para la persona que ya tiene experiencia programando usando traits con clases).

## Modificación de traits en tiempo de ejecución

Queremos agregar la posibilidad de modificar nuestros traits en tiempo de ejecución, de manera similar a lo que nos permite hacer Ruby con las clases y los módulos[3]. En particular, queremos tener:

- la posibilidad de definir métodos nuevos en un trait, por ejemplo:

```
trait Atacante do
    def ataque
        10
    end
end

Atacante.define_method(:ataque_potenciado) { ataque * 2 }

class Guerrero
    uses Atacante
end
un_guerrero = Guerrero.new

un_guerrero.ataque_potenciado #=> 20
```

[3] En verdad, este tendría que ser un punto obligatorio en toda implementación productiva de traits (u otros elementos de programación) en Ruby. No tenemos que olvidarnos de que es gracias a este tipo de herramientas reflexivas que estamos pudiendo programar nuestros traits en primer lugar!

- al igual que en las clases y en los mixines, *esto debería funcionar retroactivamente*, incluso después de haber usado el trait:

```
class Guerrero
    uses Atacante
end
un_guerrero = Guerrero.new

Atacante.define_method(:ataque_potenciado) { ataque * 2 }

un_guerrero.ataque_potenciado #=> 20
```

- y también debería funcionar con traits resultantes de operaciones:

```
trait Defensor do
    def defensa
        5
    end
end
AtacanteYDefensor = Atacante + Defensor
class Guerrero
    uses AtacanteYDefensor
end
un_guerrero = Guerrero.new

Atacante.define_method(:ataque_potenciado) { ataque * 2 }

un_guerrero.ataque_potenciado #=> 20
```

# 📎 Anexo: cómo hacer algunas cosas con Ruby

## Definir una constante dinámicamente

Pueden definirse constantes dinámicamente usando el mensaje `const_set(constante, objeto)`.

Ejemplo:

```
n = 40
nombre = :ObjetoMagico
Object.const_set(nombre, n)
# o, alternativamente, Object.const_set(:ObjetoMagico, 40)

ObjetoMagico + 2 # => 42
```

Para más información, pueden consultar la documentación de Ruby[4].

## Hacer algo si no se encuentra una constante

Ruby envía el mensaje `const_missing` cada vez que se intenta acceder a una constante que no está definida, pasándole el nombre de la constante como argumento, y el resultado se retornará como el valor de la constante que no se había encontrado. Si redefinimos el método `const_missing`, podemos implementar un comportamiento personalizado que se ejecutará cuando esto suceda. Pueden leer más en la documentación de Ruby[5].

Por ejemplo, para retornar 2 como valor de todas las constantes que sean referenciadas sin haberse definido anteriormente, podemos escribir:

```
def Object.const_missing(name)
   2
end
```

## Comportamiento personalizado para operadores

En Ruby, cuando hacemos `2 + 3`, estamos enviando un mensaje como cualquier otro (en este caso el mensaje `+`, enviado al objeto `2` y el objeto `3` como argumento). Por lo tanto, si queremos que nuestros objetos se comporten de una manera particular al recibir estos mensajes sólo debemos definir los métodos correspondientes.

[4] https://docs.ruby-lang.org/en/3.4/Module.html#method-i-const_set
[5] https://docs.ruby-lang.org/en/3.4/Module.html#method-i-const_missing

Ejemplo:

```
class Point
    attr_reader :x, :y

    def initialize(x, y)
        @x = x
        @y = y
    end

    def +(aPoint)
       self.class.new(@x + aPoint.x, @y + aPoint.y)
    end
end

p1 = Point.new(1, 2)
p2 = Point.new(3, 4)

p3 = p1 + p2
p3.x # => 4
p3.y # => 6
```